

StudyBuddy AI

Project Report — Vibe Coding: Building & Deploying an AI Web Application

Team Leader	Sharoon Kiran P
Team Member	Nikhil Thomas
Live Application URL	https://iloveaistudyhub-olc5qh0vm-nikhils-projects-77475fa8.vercel.app/

1. Application Overview and Tech Stack

StudyBuddy AI is a full-stack web application that generates custom multiple-choice quizzes on any topic using a large language model. A user enters a topic and selects a difficulty level (Beginner, Intermediate, or Advanced), and the backend prompts an LLM to produce exactly five multiple-choice questions, which the frontend renders as an interactive quiz with instant scoring and an answer-review walkthrough. The project also includes a companion research-summary module for structured topic research.

Layer	Technology
Frontend	React 19, TypeScript, Vite, Tailwind CSS — deployed on Vercel
Backend	Node.js, Express.js — deployed on Render
Database	MongoDB (MongoDB Atlas, free-tier M0 cluster)
LLM Integration	Google Gemini 2.5 Flash via OpenRouter API, using the Vercel AI SDK's <code>generateText</code>
Version Control	Git and GitHub, with <code>.env</code> -based secret management

2. Prompting Strategy and Frameworks Used

Two layers of prompting were used in this project: prompts written into the application itself (to instruct the LLM how to generate quiz content), and prompts used to direct an AI coding assistant (Claude) to build, debug, and deploy the application. The overall approach followed an **error-driven, iterative debugging loop**: reproduce the problem, paste the exact error message or screenshot to the assistant, get a root-cause diagnosis and fix, verify, then move to the next issue — rather than trying to design the whole system perfectly up front.

2.1 In-application LLM prompt (quiz generation)

The backend's `quiz.service.js` sends a structured system + user prompt to Gemini 2.5 Flash that: (1) states the topic and difficulty explicitly, (2) requires exactly 5 questions with exactly 4 options each, (3) forces the output into a strict JSON schema, and (4) explicitly instructs the model that the topic can be from any subject area — not just software or technical topics — so quiz quality does not degrade for non-technical subjects.

"You are a subject-matter quiz writer covering all fields of knowledge, not just software topics. Output strictly valid JSON conforming to the requested schema, with no markdown code fences and no commentary before or after the JSON."

2.2 Development-assistant prompting (sample prompts used)

- *"find what the problem is, the frontend is working but its quiz generator kind of thing not working"* — an intentionally under-specified, symptom-first prompt used to trigger root-cause investigation rather than guesswork.
- *"after adding .env to github it is backend gitignore idk why"* — a confusion-report prompt used to get a plain-language explanation of Git/.gitignore behavior.
- *"the vercel deployment is now wasted, not even loading frontend"* — used to diagnose a monorepo root-directory misconfiguration.
- *"can give better questions about anything, make the code right"* — a quality-improvement prompt used to identify and fix a hardcoded, domain-specific fallback question pool.

This reflects a deliberate strategy of writing prompts the way a real user would report a problem (symptoms and screenshots first) rather than pre-diagnosing the issue, then letting the assistant reproduce the bug in an isolated environment before proposing a fix.

3. Phase-by-Phase Development Summary

Phase 1 — Initial scaffolding

Frontend (React/TypeScript/Vite/Tailwind) and backend (Express/MongoDB/OpenRouter) were scaffolded, including the QuizGenerator and ResearchService components and the corresponding Express routes/controllers/services.

Phase 2 — Local debugging

Diagnosed why the quiz generator failed while the frontend still loaded: the backend was calling `process.exit(1)` on startup whenever `MONGODB_URI` was missing, because `.env` is (correctly) excluded via `.gitignore` and therefore not present after a fresh clone or deploy.

Phase 3 — Version control setup

Initialized Git, resolved 'remote origin already exists' and 'fetch first' rejected-push errors, and verified `.env` was never tracked or pushed to GitHub, protecting API keys and database credentials.

Phase 4 — Frontend deployment (Vercel)

Diagnosed a blank/failed Vercel deployment caused by the repository being a monorepo (frontend/ and backend/ as sibling folders) without a Root Directory setting; fixed by setting Root Directory to 'frontend' in Vercel project settings.

Phase 5 — Backend deployment (Render) and database (MongoDB Atlas)

Provisioned a MongoDB Atlas free-tier cluster with network access opened for cloud hosts, then deployed the Express backend to Render as a Web Service with Root Directory 'backend' and `MONGODB_URI` / `OPENROUTER_API_KEY` set as environment variables in Render's dashboard rather than committed to the repo.

Phase 6 — Frontend/backend wiring and security hardening

Removed a leftover client-side 'auto-detect backend port' routine that scanned localhost:3000/5000 directly from the deployed site — this was triggering Chrome's Local Network Access permission prompt on production. Replaced it with a single VITE_API_URL environment variable, set in Vercel, pointing at the live Render backend URL.

Phase 7 — Output quality improvement

Identified that the AI-generation fallback question pool (used only if the LLM call fails) was hardcoded with software/programming-specific content (npm, package.json, React) that produced nonsensical questions for non-technical topics. Rewrote the fallback to be fully topic-agnostic, and hardened the AI response JSON parser to strip stray text/markdown fences and validate question structure before use.

4. Application Architecture

The application follows a standard three-tier architecture, deployed across two separate hosting platforms connected via environment-configured URLs:

```
React frontend (Vercel) → HTTPS/REST (VITE_API_URL) → Express backend (Render) → MongoDB Atlas (data) + OpenRouter → Gemini 2.5 Flash (LLM)
```

- **Frontend:** React components (QuizGenerator, Dashboard, Sidebar, Navbar) call a thin service layer (quizService.ts, researchService.ts) built on Axios, which reads the backend base URL from the VITE_API_URL build-time environment variable.
- **Backend:** Express routers (/api/quiz, /api/decks, /api/flashcards, /api/research-results) delegate to controllers, which call service modules responsible for LLM prompting (quiz.service.js) and MongoDB persistence via Mongoose models.
- **LLM layer:** config/models.js wraps the OpenRouter-compatible OpenAI SDK client to call google/gemini-2.5-flash, with a custom fetch wrapper to normalize token-limit parameters.
- **Secrets:** MONGODB_URI and OPENROUTER_API_KEY exist only as environment variables — locally in a gitignored .env file, and in production inside Render's environment variable dashboard — never in frontend code or version control.

5. Challenges Encountered and How They Were Resolved

Backend crashing silently on missing MONGODB_URI

app.js called process.exit(1) if MONGODB_URI was undefined, which happened immediately after cloning the repo since .env is gitignored by design. This made it look like only the quiz feature was broken, when in fact the entire backend process was exiting before Express started listening. Resolved by creating a proper local .env and, for deployment, setting the same variable directly in Render's dashboard instead of relying on a committed file.

Confusion about .env not appearing on GitHub

Believing .env should be visible in the pushed repository after committing. Clarified that backend/.gitignore intentionally excludes .env from version control for security, and that this is expected, correct behavior — not a bug — with the real fix being to configure secrets directly in each hosting platform's dashboard.

Vercel deployment showing a blank page

The repository is a monorepo with frontend/ and backend/ as sibling folders and no root-level package.json. Vercel's default Root Directory (empty, i.e. repo root) meant it could not find a buildable project. Fixed by explicitly setting Root Directory to 'frontend' in Vercel project settings and redeploying without cache.

Git 'remote origin already exists' and 'rejected — fetch first' errors

Occurred when reusing a Git working directory across multiple new GitHub repositories. Resolved using git remote set-url origin <new-repo-url> instead of git remote add, and, where the new remote repo was freshly created and empty, using a safe force-push (git push -u origin main --force).

Chrome 'Local Network Access' permission popup on the live site

The deployed frontend was still running leftover development code that probed http://localhost:3000/health and http://localhost:5000/health directly from the browser, which Chrome now gates behind a permission prompt when a public website tries to reach local-network addresses. Removed the port-scanning logic entirely and replaced it with a single VITE_API_URL environment variable read at build time.

Irrelevant fallback quiz questions for non-technical topics

When the LLM call failed (e.g. missing API key), the app fell back to a hardcoded question pool written for software-development topics (e.g. asking about package.json or npm init), which was nonsensical for topics like history or biology. Rewrote the fallback pool to be topic-agnostic and added stricter JSON extraction/validation to reduce how often the fallback is needed at all.

6. Key Learnings and Reflection

- Environment-variable management is the single most common source of "works locally, broken in production" bugs — secrets must be deliberately re-configured per hosting platform, not just committed once.
- Monorepo deployments (frontend + backend in one repository) require explicitly telling each hosting platform which subdirectory to build from; this is easy to overlook when a project "just works" locally.
- Defensive fallback logic (like the quiz question pool) needs to be designed for the general case from the start — a fallback written with only one use case in mind (programming topics) silently degrades the product for every other use case.
- Browser security features (like Chrome's Local Network Access prompts) can surface leftover development-only code that would otherwise go unnoticed until a real user sees a confusing permission dialog in production.

- An error-driven, iterative prompting style — reporting symptoms and exact error text rather than guessing at causes — allowed each issue to be reproduced and root-caused individually instead of accumulating several overlapping, harder-to-diagnose problems.

Note on Assignment Requirements Alignment

The current deployment uses Vercel (frontend) and Render (backend), which satisfies a working, publicly accessible full-stack AI application with secure secret management. The assignment brief specifically requires deployment to AWS (App Runner or Elastic Beanstalk), Docker containerization with a working Dockerfile, and streaming/progressive LLM responses. These three requirements are not yet met in the current build and should be addressed — containerizing the backend, deploying that container to AWS App Runner, and converting the LLM call from `generateText` to a streaming equivalent — before final submission if full credit on the Cloud Deployment and Technical Implementation criteria is required.